



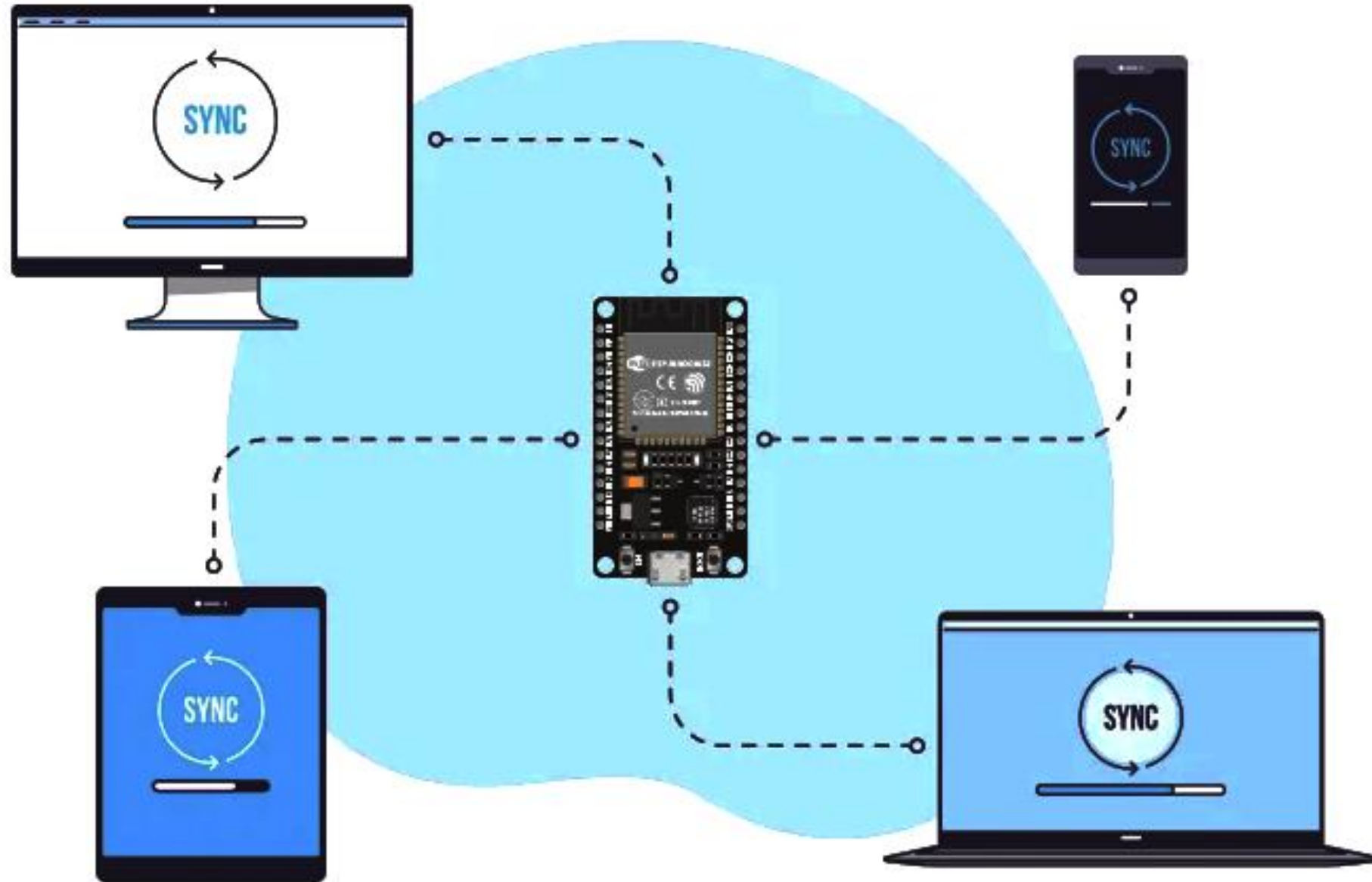
Naan Mudhalvan

Industrial IoT and Industry 4.0

Tektork Private Limited, Coimbatore

ESP32 – WebSocket

Web Socket (WebServer with WebSocket)



Web Socket

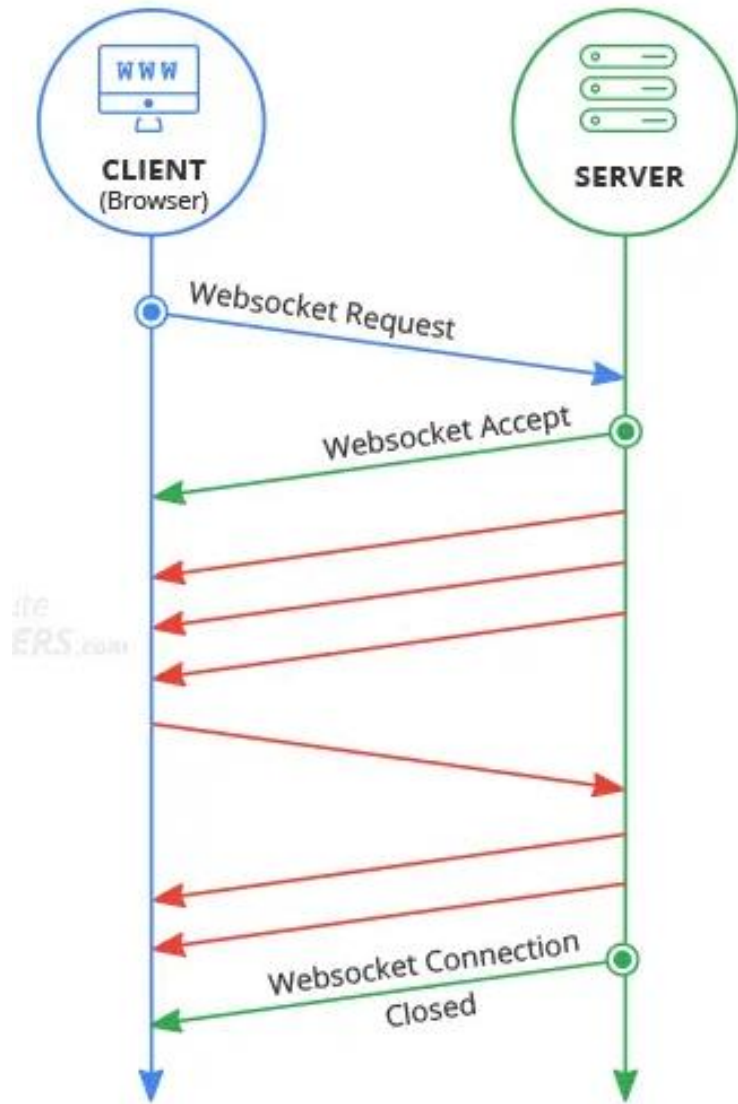
- WebSocket is the name of a communication protocol that enables bi-directional (more precisely, full-duplex) communication between a client and a web server.
- Simply put, WebSocket is a technology that allows both the client and the server to establish a connection through which either side can send messages to the other at any time.
- This differs from a regular HTTP connection, where the client initiates a request, the server sends a response, and then the connection terminates.
- In fact, WebSocket is a completely different communication protocol—instead of repeatedly opening and closing connections, once a client establishes a WebSocket connection with a server, both ends can send and receive data continuously.

Web Socket cond..

- While the server listens for new messages, all connected clients are also actively listening.
- As a result, the server can send data to a specific client or broadcast it to all clients without requiring a new request.
- Moreover, the connection stays alive until either the client or the server closes it, allowing for ongoing, real-time communication between them.

Web Socket cond..

WebSocket



Benefits of Web Socket

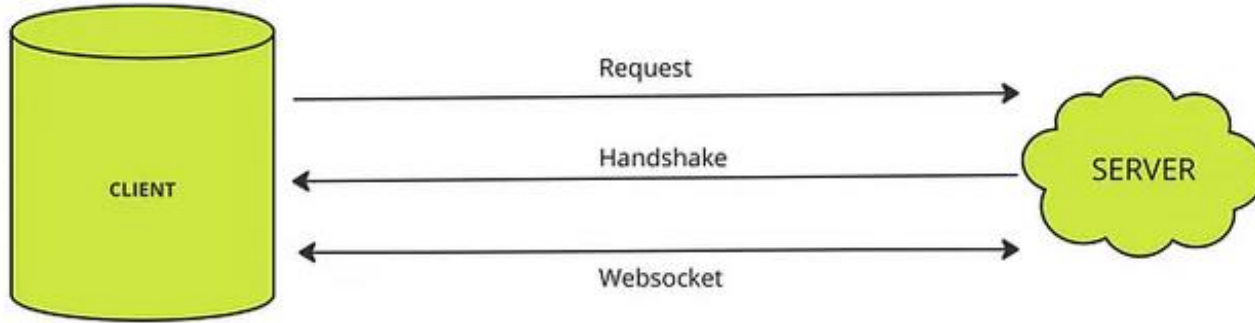
- **Real-Time Updates:** Automatically push sensor data to clients without manual refreshing .
- **Multi-Client Synchronization:** Broadcast data to all connected devices simultaneously (e.g., multiple browser tabs or devices) .
- **Efficiency:** Reduced latency and network overhead compared to HTTP polling .
- **Bidirectional Communication:** Clients can send commands (e.g., toggle GPIO) while receiving live data .

Web Socket Working Principle

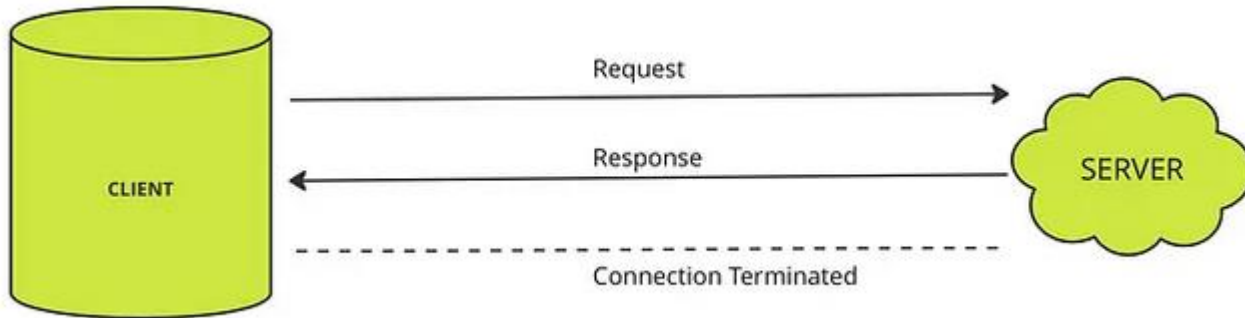
It starts with a normal HTTP connection and then upgrades to the WebSocket protocol:

- **Connection Start** – The client sends an HTTP request with the Upgrade: websocket header to the server.
- **Handshake** – The server responds with status **101 Switching Protocols** to confirm the upgrade.
- **Persistent TCP Link** – The connection switches from HTTP to the WebSocket protocol and stays open.
- **Full-Duplex Communication** – Both client and server can send messages anytime without waiting for requests.
- **Data Frames** – Messages are split into frames (text or binary) and sent over the connection.
- **Low Overhead** – There are no repeated HTTP headers; only message data is exchanged.
- **Keep-Alive** – The connection remains active until either side closes it.
- **Close Handshake** – One side sends a close frame; the other replies, then the TCP connection closes.

WEBSOCKET - HTTP



Once the handshake between client and server is established, connection remain persistent and last until any parts disconnects (**WEBSOCKET**)



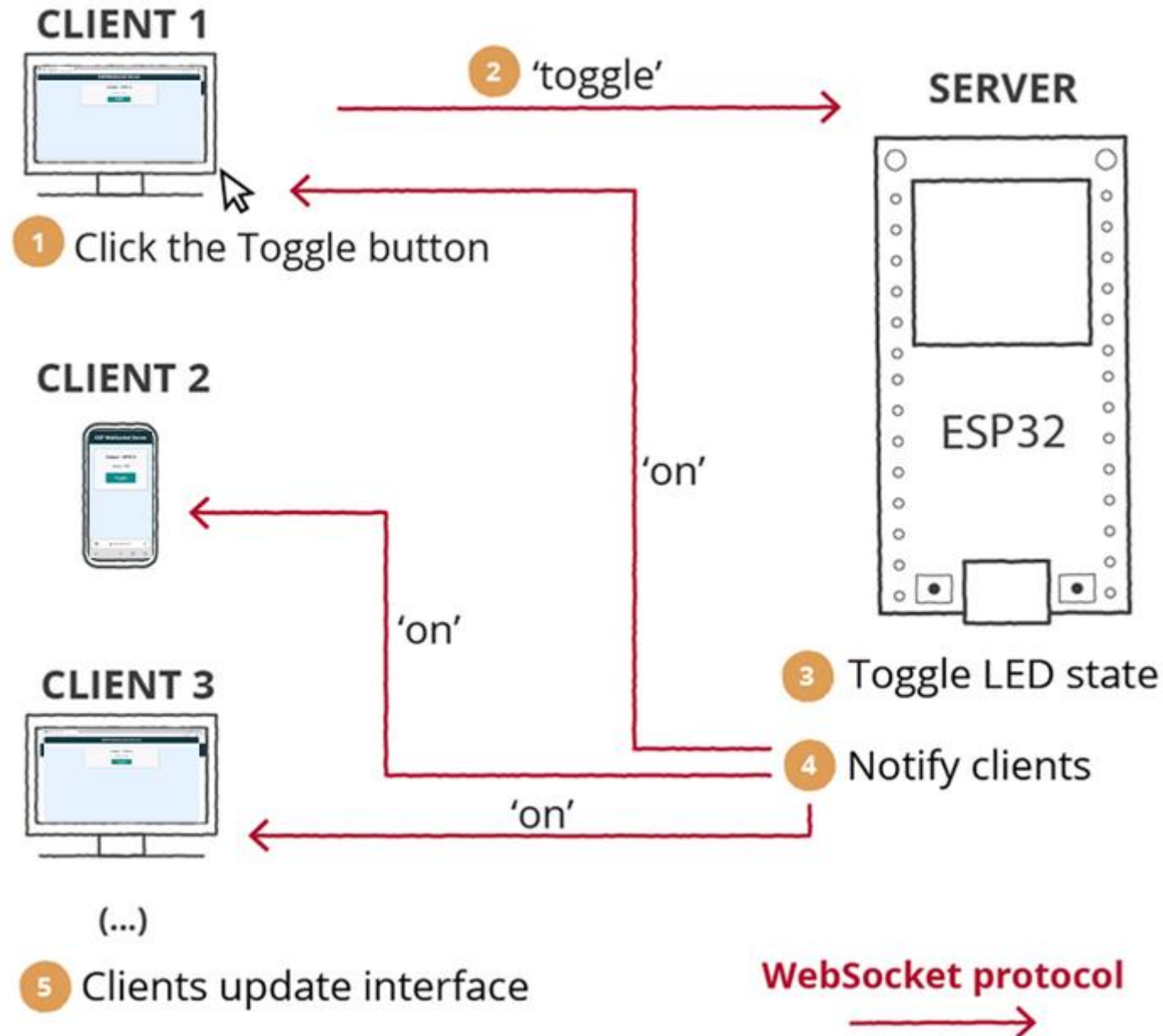
Every request with result in new TCP connection and after the response is received , connection will be terminated(**HTTP**)

Feature	HTTP	WebSocket
Connection Model	Request-Response (Unidirectional)	Persistent, Full-Duplex (Bidirectional)
Connection Duration	Short-lived, per request	Long-lived, persistent
Communication Direction	Client to Server	Both Client and Server
Protocol Overhead	High (headers per request)	Low (minimal framing after handshake)
State	Stateless	Stateful
Data Format	Primarily Text (HTML, JSON, XML)	Text and Binary
Real-time Capability	Limited (polling needed)	Native real-time communication
Connection Establishment	New connection for each request	Initial HTTP handshake, then upgraded connection
Use Cases	Web page loading, REST APIs, form submission	Real-time chat, gaming, live updates
Caching	Supported	Not supported
Browser Support	Universal	Broad in modern browsers
Security	HTTPS	WSS (secure WebSocket)
Resource Usage	Low per session	Higher due to persistent connections
Scalability	High (stateless)	Needs careful connection management

Explain the WebSocket LED toggle.

- The ESP32 web server displays a web page with a button to toggle the state of GPIO 4.
- For simplicity, we are controlling GPIO 2, which is the on-board LED. You can use this example to control any other GPIO pin.
- The interface shows the current state of the GPIO pin. Whenever the GPIO state changes, the interface updates instantly.
- The GPIO state is automatically updated on all connected clients. This means that if you have multiple browser tabs open on the same device or on different devices, they will all reflect the changes simultaneously.

The following image illustrates what happens when you click the “Toggle” button



Here's what happens when you click the **“Toggle”** button:

- You click the **“Toggle”** button on the web page.
- The client (your browser) sends a message via the WebSocket protocol with the content "toggle".
- The ESP32 (server) receives this message and knows it should toggle the LED state. If the LED was previously off, it turns it on, and vice versa.
- The ESP32 then sends the new LED state to all connected clients using the WebSocket protocol.
- All clients receive this message and update the LED status on their web pages accordingly, allowing all connected clients to stay in sync almost instantly when the LED state changes.

Library Installation

You can install the required libraries using the **Arduino Library Manager**:

- Open the **Library Manager** by clicking the **Library** icon on the left sidebar in the Arduino IDE.
- In the search bar, type **ESPAsyncWebServer**.
- Locate **ESPAsyncWebServer** by **ESP32Async** and click **Install**.

Next, install the **AsyncTCP** library:

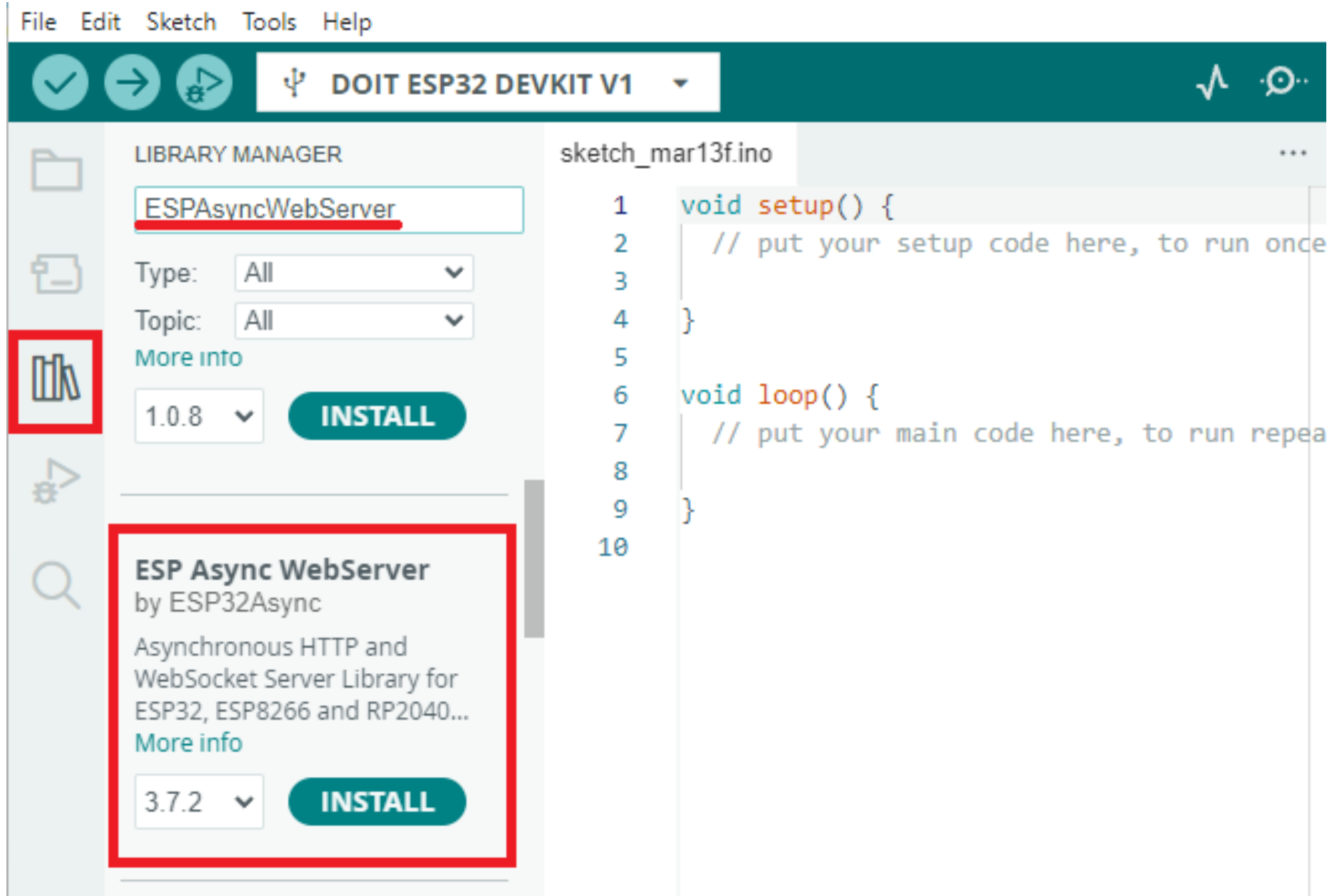
- Open the **Library Manager** in the Arduino IDE.
- Search for **AsyncTCP**.
- Find **AsyncTCP** by **ESP32Async** and click **Install**.

Library Installation Cond..

Alternatively, you can download the library files from the link below. Click “**Download ZIP**”, then install it in the Arduino IDE by going to **Sketch > Include Library > Add .ZIP Library...**

- [ESPAsyncWebServer](#) by ESP32Async
- [AsyncTCP](#) by ESP32Async

ESPAsyncWebServer Installation



File Edit Sketch Tools Help

DOIT ESP32 DEVKIT V1

LIBRARY MANAGER

ESPAsyncWebServer

Type: All

Topic: All

More info

1.0.8

INSTALL

ESP Async WebServer
by ESP32Async

Asynchronous HTTP and WebSocket Server Library for ESP32, ESP8266 and RP2040...

More info

3.7.2

INSTALL

sketch_mar13f.ino

```

1 void setup() {
2     // put your setup code here, to run once
3
4 }
5
6 void loop() {
7     // put your main code here, to run repea
8
9 }
10

```


AsyncTCP Installation

File Edit Sketch Tools Help





DOIT ESP32 DEVKIT V1

LIBRARY MANAGER

Type: All

Topic: All



AsyncTCP by dvarrel
 Async TCP Library for ESP32.
 Forked from
<https://github.com/me-no-...>
[More info](#)
 1.1.4  **INSTALL**

Async TCP by ESP32Async
 Async TCP Library for ESP32
 Async TCP Library for ESP32
[More info](#)
 3.3.6  **INSTALL**

sketch_mar13d.ino

```

1 void setup() {
2     // put your setup code here, to run once:
3
4 }
5
6 void loop() {
7     // put your main code here, to run repeatedly:
8
9 }
10

```

Controlling the LED

- After installing the required libraries, open the sketch, enter your Wi-Fi **SSID** and **password** in the appropriate variables, then upload the code to your ESP32 board.
- After uploading the code, open the **Serial Monitor** at the correct baud rate (usually 115200). Once the ESP32 connects to your Wi-Fi network, it will print the **IP address** assigned by your router. You can use this IP address to access the web interface from your browser.
- When the web interface is opened on multiple devices, pressing the **Toggle** button on one device sends a message to the server, which then broadcasts the updated LED state to all connected clients. As a result, all devices update their interface in real time.

Finding the ESP32 IP Address

- After uploading the code, open the **Serial Monitor** in the Arduino IDE and set the baud rate to **115200**.
- The ESP32 will connect to your Wi-Fi network and display its assigned IP address in the Serial Monitor.



- Press the ESP32 EN button (reset).
- The ESP32 connects to Wi-Fi, and outputs the ESP IP address on the Serial Monitor. Copy that IP address, because you need it to access the ESP32 WebSocket Server.

Serial Monitor Output

Output
Serial Monitor
X

Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM5')
No Line Ending
115200 baud

```

rst:0x3 (SW_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
config: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4744
load:0x40078000,len:15672
load:0x40080400,len:3152
entry 0x4008059c
Connecting to Pradeep

WiFi connected.
IP address:
192.168.214.77

```

Accessing the WebSocket Server

- To access the web server, open your browser and enter the ESP32's IP address in the address bar.
- For example, in our case, the IP address is **192.168.214.77**. This will load the web interface hosted by the ESP32.

- The web server you'll build controls two LEDs connected to **GPIO 4**.
- You can access the ESP32 web server by typing its **IP address** into a browser connected to the same local network.
- By clicking the **Toggle Buttons** on the web interface, you can instantly change the state of the LED to all the clients connected

Note:

This board has two controllable LEDs: one on GPIO 33 and another on GPIO 4. You can choose to use either of them via the web interface.

Web Socket Code

```
// Import required libraries
#include <WiFi.h>                // WiFi support for ESP32
#include <AsyncTCP.h>            // Async TCP library required for AsyncWebServer
#include <ESPAsyncWebServer.h>    // Asynchronous Web server library

// Replace with your network credentials
const char* ssid = "SSID";      // Wi-Fi SSID
const char* password = "Password"; // Wi-Fi Password

bool ledState = 0;              // Variable to store the LED state (0 = OFF, 1 = ON)
const int ledPin = 4;          // Define GPIO pin 4 for LED

// Create AsyncWebServer object on port 80
AsyncWebServer server(80);

// Create WebSocket object, accessible at endpoint "/ws"
AsyncWebSocket ws("/ws");
```

```
// HTML/JS code stored in program memory for serving webpage
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE HTML><html>
<head>
  <title>ESP Web Server</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" href="data:,">
  <style>
    /* Page styling */
    html {
      font-family: Arial, Helvetica, sans-serif;
      text-align: center;
      background-color: #e6f2ff; /* Light blue background */
    }
    h1 {
      font-size: 1.8rem;
      color: white;
    }
    h2{
      font-size: 1.5rem;
      font-weight: bold;
      color: #143642;
    }
  </style>
</head>
<body>
  <div>
    <h1>ESP Web Server</h1>
    <h2>ESP Web Server</h2>
  </div>
</body>
</html>
)
```

```
.topnav {  
  overflow: hidden;  
  background-color: #143642;  
}  
body {  
  margin: 0;  
  background-color: #e6f2ff;  
}  
.content {  
  padding: 30px;  
  max-width: 600px;  
  margin: 0 auto;  
}  
.card {  
  background-color: #F8F7F9;  
  box-shadow: 2px 2px 12px 1px rgba(140,140,140,.5);  
  padding-top:10px;  
  padding-bottom:20px;  
}
```



```
.button {  
  padding: 15px 50px;  
  font-size: 24px;  
  text-align: center;  
  outline: none;  
  color: #fff;  
  background-color: #0f8b8d;  
  border: none;  
  border-radius: 5px;  
  user-select: none;  
}  
.button:active {  
  background-color: #0f8b8d;  
  transform: translateY(2px);  
}  
.state {  
  font-size: 1.5rem;  
  color: #8c8c8c;  
  font-weight: bold;  
}
```

```
</style>
</head>
<body>
  <!-- Web page header -->
  <div class="topnav">
    <h1>ESP WebSocket Server</h1>
  </div>
  <div class="content">
    <div class="card">
      <h2>Output - GPIO 4</h2>
      <!-- State of LED displayed here -->
      <p class="state">state: <span id="state">%STATE%</span></p>
      <!-- Toggle button -->
      <p><button id="button" class="button">Toggle</button></p>
    </div>
  </div>
```

```
<script>
  // WebSocket communication endpoint
  var gateway = `ws://${window.location.hostname}/ws`;
  var websocket;

  // Run when page is loaded
  window.addEventListener('load', onLoad);

  // Initialize WebSocket connection
  function initWebSocket() {
    console.log('Trying to open a WebSocket connection...');
    websocket = new WebSocket(gateway);
    websocket.onopen      = onOpen;
    websocket.onclose     = onClose;
    websocket.onmessage   = onMessage; // Receive messages
  }

  // When connection opens
  function onOpen(event) {
    console.log('Connection opened');
  }
```

```
// When connection closes, retry in 2 sec
function onClose(event) {
    console.log('Connection closed');
    setTimeout(initWebSocket, 2000);
}

// Handle incoming WebSocket messages from ESP32
function onMessage(event) {
    var state;
    if (event.data == "1"){
        state = "ON";
    }
    else{
        state = "OFF";
    }
    document.getElementById('state').innerHTML = state;
}

// On load, initialize websocket + button
function onLoad(event) {
    initWebSocket();
    initButton();
}
```

```
// Button click triggers toggle message
function initButton()
{
    document.getElementById('button').addEventListener('click', toggle);
}

// Send "toggle" command over WebSocket to ESP32
function toggle()
{
    websocket.send('toggle');
}
</script>
</body>
</html>
)rawliteral";
```

```
// Send LED state to all connected WebSocket clients
```

```
void notifyClients() {  
    ws.textAll(String(ledState));  
}
```

```
// Handle received WebSocket messages
```

```
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {  
    AwsFrameInfo *info = (AwsFrameInfo*)arg;  
    // Verify message integrity  
    if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {  
        data[len] = 0;    // Null terminate the string  
        // If toggle command received, flip LED state  
        if (strcmp((char*)data, "toggle") == 0) {  
            ledState = !ledState;  
            notifyClients(); // Notify all clients of the new state  
        }  
    }  
}
```

```
// Event handler for WebSocket events
void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client,
             AwsEventType type, void *arg, uint8_t *data, size_t len) {
    switch (type) {
        case WS_EVT_CONNECT:
            Serial.printf("WebSocket client #%u connected from %s\n",
                          client->id(), client->remoteIP().toString().c_str());
            break;
        case WS_EVT_DISCONNECT:
            Serial.printf("WebSocket client #%u disconnected\n", client->id());
            break;
        case WS_EVT_DATA:
            handleWebSocketMessage(arg, data, len);
            break;
        case WS_EVT_PONG:
        case WS_EVT_ERROR:
            break;
    }
}
```

```
// Initialize WebSocket server
void initWebSocket() {
    ws.onEvent(onEvent);
    server.addHandler(&ws);
}

// Replace HTML placeholder with actual state value
String processor(const String& var){
    Serial.println(var);
    if(var == "STATE"){
        if (ledState){
            return "ON";
        }
        else{
            return "OFF";
        }
    }
    return String();
}
```



```
void setup(){  
  // Serial port for debugging  
  Serial.begin(115200);  
  pinMode(ledPin, OUTPUT);  
  digitalWrite(ledPin, LOW); // Turn off LED initially  
  
  // Connect to Wi-Fi network  
  WiFi.begin(ssid, password);  
  while (WiFi.status() != WL_CONNECTED)  
{  
    delay(1000);  
    Serial.println("Connecting to WiFi..");  
  }  
  
  // Print ESP Local IP Address  
  Serial.println(WiFi.localIP());  
  
  // Initialize WebSocket  
  initWebSocket();  
}
```

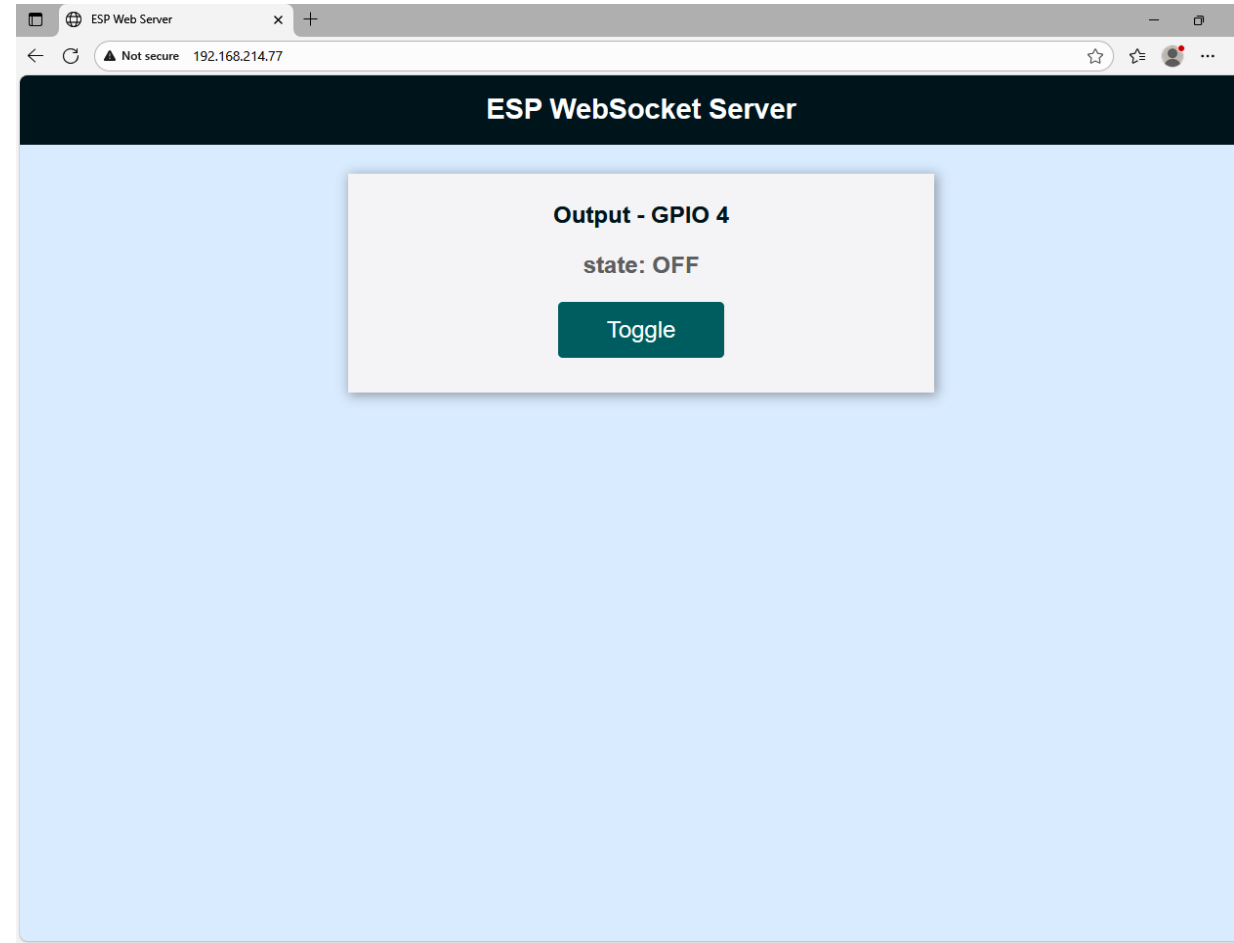
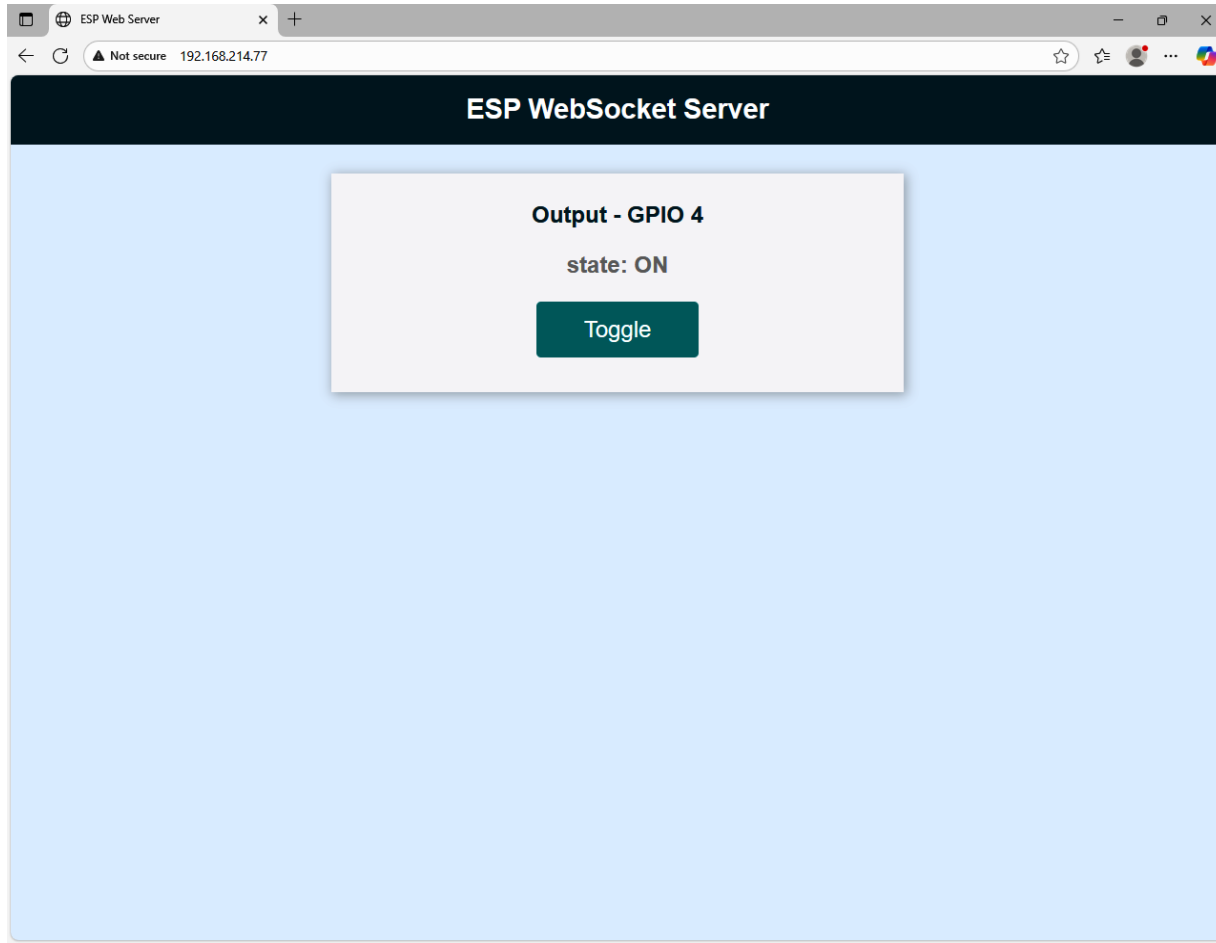
```
// Define root route "/"
server.on("/", HTTP_GET, [](AsyncWebServerRequest *request)
{
    // Send HTML page with processor function replacing placeholders
    request->send(200, "text/html", index_html, processor);
});

// Start HTTP server
server.begin();
}

void loop()
{
    // Periodically clean disconnected WebSocket clients
    ws.cleanupClients();

    // Set LED pin based on ledState value
    digitalWrite(ledPin, ledState);
}
```

Local ESP32 Web Socket Server interface



Thank You